Анализ данных с использованием языка Python

Пояснения к коду. Семинар 10

Предобработка данных: выбросы и пропуски

А. Н. Лата

Пояснения к ноутбуку `notebooks/seminar-10-outliers-missing.ipynb`. Номер ячейки — её порядковый номер среди ячеек кода ноутбука, считая сверху. Для каждой ячейки приведён её код с номерами строк, а под ним — что делает каждая команда и что означает каждый аргумент именно в этой ячейке. Команды, разобранные в пояснениях к семинарам 8 и 9, здесь только называются. Код совпадает с ноутбуком, только пробелы перед комментариями сокращены. Полные подписи функций со всеми параметрами — в справочнике `reference/pandas.md`, раздел «Выбросы и аномалии».

1 Ошибка, редкость или код

Ячейка 1. Таблица ДТП и сводка четырёх счётчиков

```
1 import matplotlib.pyplot as plt
2 import numpy as np
3 import pandas as pd
4 import seaborn as sns
5
6 sns.set_theme(style='whitegrid')  # оформление графиков по умолчанию
7 dtp = pd.read_csv('../datasets/legal/dtp-bryansk.csv', index_col='id', ↩
  ↪ parse_dates=['datetime'])  # ДТП Брянской области
8 counts = ['participants_count', 'vehicles_count', 'dead_count', ↩
  ↪ 'injured_count']
9 dtp[counts].describe().round(2)  # сводка четырёх счётчиков
```

– Строки 1–7 — подключение и таблица, как на семинарах 8 и 9; plotly и папка для файлов сегодня не нужны.

– `counts = [...]` — список имён четырёх столбцов-счётчиков; он нужен в нескольких ячейках, поэтому сохранён в переменную.

– `dtp[counts].describe()` — сводка каждого из четырёх столбцов: число значений, среднее, стандартное отклонение, минимум, квартили 25%, 50%, 75% и максимум. `.round(2)` — округлить до двух знаков.

Ячейка 2. Три ДТП с наибольшим числом погибших

```
1 dtp.nlargest(3, 'dead_count')[['datetime', 'district', 'category'] + ↩
  ↪ counts]  # самые тяжёлые ДТП
```

– `dtp.nlargest(3, 'dead_count')` — три строки с наибольшими значениями столбца `dead_count`.

– `[['datetime', 'district', 'category'] + counts]` — какие столбцы показать. Знак `+` между двумя списками склеивает их в один: три столбца и четыре счётчика.

Ячейка 3. В чём ехали участники самого тяжёлого ДТП

```
people = pd.read_csv('../datasets/legal/dtp-bryansk-participants.csv')  # участники ДТП
people[people['dtp_id'] == 28405]['vehicle_category'].value_counts()  # классы машин участников
```

– `people[people['dtp_id'] == 28405]` — участники ДТП номер 28405, самого тяжёлого из ячейки 2.

– `['vehicle_category'].value_counts()` — сколько участников ехало в транспорте каждого класса.

Ячейка 4. Таблица машин и возраст машины

```
people = people.merge(dtp[['datetime']], left_on='dtp_id', right_index=True)  # дата ДТП каждому участнику
vehicles = people.dropna(subset=['vehicle_no']).drop_duplicates(subset=['dtp_id', 'vehicle_no'])  # строка на машину
vehicles['vehicle_age'] = vehicles['datetime'].dt.year - vehicles['vehicle_year']  # возраст машины в годах
print(len(people), len(vehicles))
vehicles['vehicle_age'].describe().round(2)
```

– `people.merge(dtp[['datetime']], ...)` — присоединяет к каждому участнику дату его ДТП. Из таблицы ДТП берётся один столбец, `dtp[['datetime']]`: двойные скобки дают таблицу, а не Series. Ключи — как на семинаре 8: `dtp_id` у участников, индекс у ДТП. Результат присваивается обратно в `people`, поэтому второй запуск ячейки присоединит дату ещё раз, столбцы станут `datetime_x` и `datetime_y`, и строка 3 упадёт с `KeyError: 'datetime'`.

– `dropna(subset=['vehicle_no'])` — убирает строки без номера машины: пешеходов и других участников без транспорта. `subset` — смотреть пропуски только в этом столбце.

– `drop_duplicates(subset=['dtp_id', 'vehicle_no'])` — оставляет первую строку каждой машины. Машину задают два столбца вместе: номер ДТП и номер машины в нём. Это не очистка, а другая таблица: строка — машина, а не человек.

– Строка 3 — возраст машины: год ДТП минус год выпуска.

– `print(len(people), len(vehicles))` — число строк в двух таблицах: `24717 15870`.

– `vehicles['vehicle_age'].describe()` — сводка возраста; максимум 2 014 лет.

Ячейка 5. Двенадцать самых старых машин

```
vehicles.nsmallest(12, 'vehicle_year')[['dtp_id', 'brand', 'vehicle_category', 'vehicle_year']]  # самые старые машины
```

– `vehicles.nsmallest(12, 'vehicle_year')` — двенадцать строк с наименьшим годом выпуска; пара к `nlargest`. Пропуски года в отбор не попадают.

– `[[...]]` — четыре столбца для показа: номер ДТП, марка, класс, год выпуска.

2 Форма распределения и правила выбросов

Ячейка 6. Возраст машин: четыре числа и два рисунка

```
age = vehicles.loc[vehicles['vehicle_year'] >= 1900, 'vehicle_age']  # ↩
↪ возраст с настоящим годом выпуска
print(age.agg(['count', 'mean', 'median', 'skew']).round(2))
fig, axes = plt.subplots(1, 2, figsize=(12, 4))
sns.histplot(x=age, discrete=True, ax=axes[0])  # гистограмма возраста
axes[0].set_title('Возраст машины')
axes[0].set_xlabel('Возраст, лет')
axes[0].set_ylabel('Число машин')
sns.boxplot(x=age, ax=axes[1])  # ящик с усами
axes[1].set_title('Возраст машины: ящик с усами')
axes[1].set_xlabel('Возраст, лет')
fig.tight_layout()
```

– `vehicles.loc[маска, 'vehicle_age']` — возраст только тех машин, у которых год выпуска не раньше 1900-го; результат — Series `age`.

– `age.agg(['count', 'mean', 'median', 'skew'])` — несколько статистик сразу: число значений, среднее, медиана, асимметрия. Асимметрия `skew` около нуля у симметричного распределения и больше нуля при длинном хвосте справа.

– Строки 3–10 — две панели на одной фигуре, как на семинаре 8: слева гистограмма, справа ящик с усами. `sns.boxplot(x=age, ...)` — ящик лежит горизонтально, потому что значения переданы как `x`.

– `fig.tight_layout()` — в конце нет точки с запятой, но печатать нечего: метод ничего не возвращает.

Ячейка 7. Правило IQR

```
q1, q3 = age.quantile([0.25, 0.75])  # первый и третий квартили
iqr = q3 - q1  # межквартильный размах
lower, upper = q1 - 1.5 * iqr, q3 + 1.5 * iqr  # границы усов ящика
print(q1, q3, iqr, lower, upper)
(age > upper).sum()  # машин старше верхней границы
```

– `age.quantile([0.25, 0.75])` — два квартиля сразу: значения, ниже которых лежат четверть и три четверти данных. Список долей пишется в долях единицы, а не в процентах. Два числа раскладываются в две переменные `q1` и `q3`, как `fig, ax` у `plt.subplots`.

– `iqr = q3 - q1` — межквартильный размах, длина ящика.

– Строка 3 — две границы одной строкой: слева две переменные, справа через запятую два выражения.

– `(age > upper).sum()` — сколько машин старше верхней границы.

Ячейка 8. Правило трёх сигм

```
1 z = (age - age.mean()) / age.std()  # z-оценка каждого значения
2 print(round(age.mean() + 3 * age.std(), 2))
3 (z.abs() > 3).sum()  # выбросов по трём сигмам
```

– `(age - age.mean()) / age.std()` — z-оценка каждой машины: на сколько стандартных отклонений её возраст отстоит от среднего. Считается для всех значений сразу.

– Строка 2 — сама граница в годах: среднее плюс три стандартных отклонения.

– `z.abs() > 3` — модуль z больше трёх: так ловятся и слишком большие, и слишком малые значения.

Ячейка 9. 99-й процентиль и три правила рядом

```
1 p99 = age.quantile(0.99)  # 99-й процентиль возраста
2 print(p99)
3 pd.Series({'IQR': (age > upper).sum(), 'три сигмы': (z.abs() > 3).sum(), ↩
↪ '99-й процентиль': (age > p99).sum()})  # три правила
```

– `age.quantile(0.99)` — значение, ниже которого лежат 99 процентов возрастов.

– `pd.Series({...})` — Series из словаря: ключи стали индексом, значения — числами. Так три результата собираются в одну подписанную табличку.

3 Где правила ломаются

Ячейка 10. Правила для четырёх счётчиков сразу

```
1 q1_c, q3_c = dtp[counts].quantile(0.25), dtp[counts].quantile(0.75)  # ↩
↪ квартили каждого счётчика
2 iqr_c = q3_c - q1_c  # размах каждого счётчика
3 z_c = (dtp[counts] - dtp[counts].mean()) / dtp[counts].std()  # z-оценки ↩
↪ всех счётчиков
4 outside = (dtp[counts] < q1_c - 1.5 * iqr_c) | (dtp[counts] > q3_c + 1.5 * ↩
↪ iqr_c)  # маска за границами IQR
5 pd.DataFrame({'q1': q1_c, 'q3': q3_c, 'выбросов IQR': outside.sum(),
6               'граница трёх сигм': dtp[counts].mean() + 3 * ↩
↪ dtp[counts].std(),
7               'выбросов трёх сигм': (z_c.abs() > 3).sum()}).round(2)
```

– `dtp[counts].quantile(0.25)` — квартиль каждого столбца таблицы: Series, где в индексе имена счётчиков. Так же `mean()` и `std()` у таблицы считаются по столбцам.

– Строки 3–4 — те же формулы, что в ячейках 7 и 8, но для таблицы. Когда таблицу сравнивают или вычитают с Series, pandas сопоставляет имена столбцов таблицы с индексом Series: у каждого столбца своя граница.

– Строка 4 — маска «ниже нижней или выше верхней границы»: каждое сравнение в скобках, между ними `|`, «или». Результат — таблица из `True` и `False`.

– `outside.sum()` — сумма по каждому столбцу: число выбросов у каждого счётчика.

– `pd.DataFrame({...})` — таблица из пяти Series с общим индексом: каждая стала столбцом.

Ячейка 11. Маска без скобок: намеренная ошибка

```
1 age[age < lower | age > upper]  # выбросы с обеих сторон
```

– Без скобок Python сначала выполняет `|`, а потом сравнения, и читает маску как `age < (lower | age) > upper`. Побитовое «или» дробного числа с Series невозможно: `TypeError: Cannot perform 'ror_' with a dtyped [float64] array and scalar of type [bool]`. Ошибка намеренная.

Ячейка 12. Та же маска со скобками

```
1 outliers = age[(age < lower) | (age > upper)]  # выбросы с обеих сторон
2 vehicles.loc[outliers.index, 'vehicle_category'].value_counts().head()  # ↩
↪ классы самых старых машин
```

– Строка 1 — правильная маска: каждое сравнение в своих скобках.

– `outliers.index` — номера строк выбросов. По этим номерам `vehicles.loc[...]` берёт из таблицы машин столбец `vehicle_category`, класс машины.

– `value_counts().head()` — пять самых частых классов.

Ячейка 13. Выбросы в сводке по дням

```
1 daily = dtp['datetime'].dt.date.value_counts()  # ДТП в каждый день
2 z_day = (daily - daily.mean()) / daily.std()  # z-оценка каждого дня
3 peaks = daily[z_day > 3]  # дни выше трёх сигм
4 print(round(daily.mean(), 2), daily.max(), daily.idxmax(), len(peaks))
5 light = dtp.loc[dtp['severity'] == 'Легкий', 'datetime'].dt.year  # годы ↩
↪ лёгких ДТП
6 pd.DataFrame({'пиковых дней': ↩
↪ pd.to_datetime(peaks.index).year.value_counts(),
7               'лёгких ДТП': ↩
↪ light.value_counts()}).fillna(0).astype(int).sort_index()  # пики и ↩
↪ учёт по годам
```

– `dt.date` — дата без времени; `value_counts()` — число ДТП в каждый день. Дни без ДТП в сводку не попадают.

– Строки 2–3 — z-оценка каждого дня и дни, где она больше трёх.

– `daily.idxmax()` — метка, у которой значение наибольшее: день с самым большим числом ДТП.

– `pd.to_datetime(peaks.index).year` — год каждого пикового дня: даты из индекса снова превращаются в даты pandas, у которых есть год.

– `.fillna(0).astype(int).sort_index()` — в годы без пиковых дней в таблице пропуск: он заменяется нулём, числа делаются целыми, годы ставятся по порядку.

4 Пять решений и их цена

Ячейка 14. Замена «на месте»: ловушка без ошибки

```
1 vehicles['vehicle_year'].replace(1, np.nan, inplace=True)  # год 1 в пропуск
2 (vehicles['vehicle_year'] == 1).sum()  # машин с годом 1
```

– `replace(1, np.nan, inplace=True)` — заменить 1 пропуском «на месте». Но столбец `vehicles['vehicle_year']` — отдельный объект, и в pandas 3 изменение на месте до таблицы не доходит: появляется предупреждение `ChainedAssignmentError`, а таблица прежняя.

– Строка 2 — проверка: машин с годом 1 по-прежнему 4.

Ячейка 15. Замена присваиванием и возраст заново

```
1 raw_age = vehicles['vehicle_age']  # возраст до замены
2 vehicles['vehicle_year'] = vehicles['vehicle_year'].replace(1, np.nan)  # ↩
↪ год 1 в пропуск
3 vehicles['vehicle_age'] = vehicles['datetime'].dt.year - ↩
↪ vehicles['vehicle_year']  # возраст заново
4 print((vehicles['vehicle_year'] == 1).sum(), vehicles['vehicle_age'].max())
```

– `raw_age = vehicles['vehicle_age']` — прежний возраст сохраняется под отдельным именем для сравнения в ячейке 18. Когда в строке 3 столбцу присваивается новый возраст, `raw_age` остаётся старым.

– Строка 2 — правильная замена: результат `replace` присваивается обратно в столбец.

– Строка 3 — возраст пересчитан: у машин без года выпуска он теперь пропуск.

– Строка 4 — проверка: машин с годом 1 ноль, самая старая машина — 62 года.

Ячейка 16. Исправление переставленных координат

```
1 frame = dtp['lat'].between(51.8, 54.2) & dtp['lon'].between(31, 35.5)  # ↩
↪ внутри рамки области
2 swapped = dtp['lat'].between(31, 35.5) & dtp['lon'].between(51.8, 54.2)  # ↩
↪ широта и долгота местами
3 print((~frame).sum(), (dtp['lat'] == 0).sum(), swapped.sum())
4 dtp.loc[swapped, 'lat'], dtp.loc[swapped, 'lon'] = dtp.loc[swapped, ↩
↪ 'lon'], dtp.loc[swapped, 'lat']  # поменять местами
5 (~(dtp['lat'].between(51.8, 54.2) & dtp['lon'].between(31, 35.5))).sum()  ↩
↪ # вне рамки после исправления
```

– `frame` — маска ДТП внутри рамки области, как на семинаре 8.

– `swapped` — маска ДТП, у которых широта попадает в рамку долготы, а долгота — в рамку широты: их записали местами.

– `~frame` — тильда переворачивает маску: ДТП вне рамки.

– Строка 4 — обмен значений, как `a, b = b, a`: справа сначала берутся долгота и широта отмеченных строк, потом слева они записываются в обратные столбцы.

– Строка 5 — сколько ДТП вне рамки после исправления: 327. Если запустить ячейку второй раз, маска `swapped` ничего не найдёт: координаты уже исправлены.

Ячейка 17. Пометка старых машин и сравнение с пропуском

```
vehicles['old_car'] = vehicles['vehicle_age'] > upper  # старше верхней границы
normal = vehicles['vehicle_age'] <= upper  # не старше границы
print(vehicles['old_car'].sum(), normal.sum(), vehicles['vehicle_age'].isna().sum(), len(vehicles))
```

– `vehicles['old_car'] = ...` — новый логический столбец: `True` у машин старше верхней границы IQR. Так редкие значения помечают, не трогая их.

– `normal` — маска «не старше границы».

– Строка 3 — старые, не старые, без возраста и все машины: `268 14578 1024 15870`. У машины без возраста обе маски `False`: сравнение с пропуском всегда ложно.

Ячейка 18. Цена каждого решения

```
age = vehicles['vehicle_age']  # возраст после замены
measures = ['mean', 'median', 'count']
pd.DataFrame({'как есть, с годом 1': raw_age.agg(measures),
              'год 1 в пропуск': age.agg(measures),
              'удалить старше границы': age[age <= upper].agg(measures),
              'срезать по 99-му процентилю': age.clip(upper=age.quantile(0.99)).agg(measures),
              'старше границы в пропуск': age.where(age <= upper).agg(measures)}).T.round(2)  # цена каждого решения
```

– `measures = [...]` — три статистики для `agg`: среднее, медиана, число значений. Список нужен и в ячейке 21.

– `age[age <= upper]` — удалить: остаются только значения не выше границы.

– `age.clip(upper=...)` — срезать: всё, что выше потолка, становится равным потолку; здесь потолок — 99-й процентиль. Такой срез называют винзоризацией.

– `age.where(age <= upper)` — заменить пропуском: где условие верно, значение остаётся, где ложно — ставится пропуск.

– `pd.DataFrame({...}).T` — таблица из пяти Series, а `.T` переворачивает её: решения становятся строками, статистики — столбцами.

5 Пропуски: не чем, а почему

Ячейка 19. Коды стажа и доля пропуска по ролям

```
people['experience_clean'] = people['experience'].replace([95, 96, 99], np.nan)  # коды стажа пропусками
print(people['experience'].isin([95, 96, 99]).sum())
pd.crosstab(people['role'], people['experience_clean'].isna(), normalize='index').round(3)  # доля пропуска по ролям
```

– Строка 1 — коды стажа пропуском, как на семинарах 8 и 9, но в таблице всех участников.

– `isin([95, 96, 99])` — маска: значение входит в список кодов; сумма — сколько строк с кодами.

– `pd.crosstab(people['role'], ...isna(), normalize='index')` — таблица: строки — роли, столбцы — есть ли пропуск стажа, `False` или `True`. `normalize='index'` делит каждую строку на её сумму: доля пропуска внутри каждой роли.

Ячейка 20. Медиана по всей таблице: ловушка без ошибки

```
1 filled = ↩
  ↪ people['experience_clean'].fillna(people['experience_clean'].median())  ↩
  ↪ # пропуски медианой во всей таблице
2 filled[people['role'] == 'Пешеход'].value_counts()  # «стаж» пешеходов
```

– `fillna(...median())` — заполнить все пропуски стажа одной медианой всей таблицы, 13 лет.

– `filled[people['role'] == 'Пешеход']` — «стаж» пешеходов после заполнения: у всех 2 901 пешехода — 13 лет. Ошибки нет, а данные испорчены.

Ячейка 21. Четыре способа обойтись с пропуском стажа водителей

```
1 drivers = people[people['role'] == 'Водитель']  # только водители
2 experience = drivers['experience_clean']
3 by_gender = ↩
  ↪ drivers.groupby('gender')['experience_clean'].transform('median')  # ↩
  ↪ медиана стажа своего пола
4 pd.DataFrame({'оставить пропуск': experience.agg(measures),
5                'медиана всех водителей': ↩
  ↪ experience.fillna(experience.median()).agg(measures),
6                'медиана своего пола': ↩
  ↪ experience.fillna(by_gender).agg(measures),
7                'ноль': experience.fillna(0).agg(measures)}).T.round(2)  # ↩
  ↪ цена каждого заполнения
```

– `drivers.groupby('gender')['experience_clean']` — стаж водителей по группам пола.

– `.transform('median')` — медиана своей группы на каждой строке: результат той же длины, что таблица водителей, и с теми же номерами строк. Обычное `.median()` дало бы одну строку на группу. У водителей без указанного пола группы нет, и у них пропуск.

– `experience.fillna(by_gender)` — пропуск каждой строки заполняется значением из той же строки `by_gender`.

– `experience.fillna(0)` — заполнение нулём, для сравнения.

– Строки 4–7 — таблица цены, как в ячейке 18.

Ячейка 22. Индикатор пропуска и здоровье водителей

```
drivers['experience_missing'] = drivers['experience_clean'].isna()  # ↩
↪ пропуск стажа у водителя
pd.crosstab(drivers['experience_missing'], drivers['health'], ↩
↪ normalize='index').round(3)  # здоровье водителей
```

– `drivers['experience_missing'] = ...isna()` — индикатор пропуска: логический столбец, `True` там, где стажа нет. Он сохраняет это и после любого заполнения.

– `pd.crosstab(..., normalize='index')` — доли состояний здоровья отдельно у водителей со стажем и без.

6 Повторы и логика

Ячейка 23. Повторяющиеся строки участников

```
repeated = people.duplicated(keep=False)  # строки в группах одинаковых
print(repeated.sum(), people.duplicated().sum())
people.loc[repeated, 'role'].value_counts()  # роли в повторах
```

– `people.duplicated(keep=False)` — маска: `True` у каждой строки, у которой есть полная копия, во всех строках группы одинаковых.

– `people.duplicated()` — без параметра `True` у всех строк группы, кроме первой: столько строк удалил бы `drop_duplicates`.

– `people.loc[repeated, 'role'].value_counts()` — роли участников в повторах.

Ячейка 24. Что сделал бы drop_duplicates со сводкой по здоровью

```
health = pd.DataFrame({'все строки': people['health'].value_counts(),
                       'после drop_duplicates': ↩
↪ people.drop_duplicates()['health'].value_counts()})  # сводка по здоровью
health['удалено'] = health['все строки'] - health['после drop_duplicates']
health
```

– `people.drop_duplicates()` — таблица без полных повторов; сама `people` не меняется, результат сразу идёт в сводку.

– `pd.DataFrame({...})` — две сводки рядом: все строки и после удаления.

– `health['удалено'] = ...` — разница двух столбцов: сколько людей каждого состояния исчезло бы.

Ячейка 25. Шесть логических проверок

```
checks = pd.Series({  # нарушений каждого правила
    'пострадавших больше участников': (dtp['dead_count'] + dtp['injured_count'] > dtp['participants_count']).sum(),
    'машин больше участников': (dtp['vehicles_count'] > dtp['participants_count']).sum(),
    'машин нет': (dtp['vehicles_count'] == 0).sum(),
    'выпуск позже года ДТП': (vehicles['vehicle_year'] > vehicles['datetime'].dt.year).sum(),
    'дата вне 2015-2025 годов': (~dtp['datetime'].dt.year.between(2015, 2025)).sum(),
    'координаты вне рамки области': (~(dtp['lat'].between(51.8, 54.2) & dtp['lon'].between(31, 35.5))).sum(),
})
print(checks)
dtp.loc[dtp['vehicles_count'] > dtp['participants_count'], 'category'].value_counts().head(3)  # где машин больше
```

– `pd.Series({...})` — Series из словаря, как в ячейке 9: ключ — название проверки, значение — число нарушений. Словарь записан на нескольких строках: Python ждёт продолжения, пока скобки не закрыты.

– Каждое значение — маска нарушений правила и её сумма. Сравнивать можно и выражения: `dtp['dead_count'] + dtp['injured_count'] > dtp['participants_count']` — пострадавших больше, чем участников.

– `~...between(2015, 2025)` — тильда переворачивает маску «внутри периода» в маску «вне периода».

– Последняя строка — виды ДТП, где машин больше, чем участников.